

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

PRACTICAL – WEEK 12

Name: S Svara Haasini

Roll No: 2520090170

Task 1: Producer-Consumer Problem Using Semaphores

Question:

Implement the classic producer-consumer problem using POSIX semaphores. Develop a bounded buffer where producers add items and consumers remove items with proper synchronization.

Source Code:

GNU nano 7.2

P12_T1.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>

#define ITEMS 20

int *buffer;
int buffer_size;

int in = 0;
int out = 0;

sem_t empty_slots;
sem_t full_slots;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg)
{
    (void)arg;

    for (int item = 1; item <= ITEMS; item++)
    {
        sem_wait(&empty_slots);

        pthread_mutex_lock(&mutex);

        buffer[in] = item;

        printf("Produced: %d\n", item);

        in = (in + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        sem_post(&full_slots);
    }

    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITEMS; i++)
    {
        sem_wait(&full_slots);
```



haasini@SSH: ~/OSSP_upload



GNU nano 7.2

P12_T1.c *

```
}
    return NULL;
}

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: %s <buffer_size>\n", argv[0]);
        return 1;
    }

    buffer_size = atoi(argv[1]);

    if (buffer_size <= 0)
    {
        printf("Buffer size must be positive.\n");
        return 1;
    }

    buffer = malloc(buffer_size * sizeof(int));

    sem_init(&empty_slots, 0, buffer_size);
    sem_init(&full_slots, 0, 0);

    pthread_t producer_thread;
    pthread_t consumer_thread;

    struct timespec start, end;

    clock_gettime(CLOCK_MONOTONIC, &start);

    pthread_create(&producer_thread, NULL,
                  producer, NULL);

    pthread_create(&consumer_thread, NULL,
                  consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1000000000.0;

    printf("\nBuffer size : %d\n", buffer_size);
    printf("Items      : %d\n", ITEMS);
    printf("Time taken   : %.6f seconds\n", time_taken);
    printf("Throughput   : %.2f items/second\n",
          ITEMS / time_taken);

    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);
    pthread_mutex_destroy(&mutex);

    free(buffer);

    return 0;
}
```

Output:

Buffer Size: 1, Items: 20

```
haasini@SSH: ~/OSSP_upload × + v
haasini@SSH:~/OSSP_upload/Practical_Week12$ gcc -Wall -Wextra P12_T1.c -o P12_T1 -pthread
haasini@SSH:~/OSSP_upload/Practical_Week12$ ./P12_T1 1
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Produced: 6
Consumed: 6
Produced: 7
Consumed: 7
Produced: 8
Consumed: 8
Produced: 9
Consumed: 9
Produced: 10
Consumed: 10
Produced: 11
Consumed: 11
Produced: 12
Consumed: 12
Produced: 13
Consumed: 13
Produced: 14
Consumed: 14
Produced: 15
Consumed: 15
Produced: 16
Consumed: 16
Produced: 17
Consumed: 17
Produced: 18
Consumed: 18
Produced: 19
Consumed: 19
Produced: 20
Consumed: 20

Buffer size : 1
Items       : 20
Time taken  : 0.005184 seconds
Throughput  : 3857.94 items/second
haasini@SSH:~/OSSP_upload/Practical_Week12$ ./P12_T1 5
```

Buffer Size: 5, Items: 20

```
Throughput : 3857.94 items/second
haasini@SSH:~/OSSP_upload/Practical_Week12$ ./P12_T1 5
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Produced: 6
Produced: 7
Produced: 8
Produced: 9
Produced: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
Produced: 11
Produced: 12
Produced: 13
Produced: 14
Produced: 15
Consumed: 11
Consumed: 12
Consumed: 13
Consumed: 14
Consumed: 15
Produced: 16
Produced: 17
Produced: 18
Produced: 19
Produced: 20
Consumed: 16
Consumed: 17
Consumed: 18
Consumed: 19
Consumed: 20

Buffer size : 5
Items : 20
Time taken : 0.003911 seconds
Throughput : 5114.41 items/second
haasini@SSH:~/OSSP_upload/Practical_Week12$ |
```

Observations:

1. Semaphores are counting synchronization primitives used for producer-consumer coordination.
2. empty_slots semaphore tracks the number of empty buffer positions.
3. full_slots semaphore tracks the number of filled buffer positions.
4. mutex protects access to the buffer from concurrent modification.
5. sem_wait() decrements the semaphore, blocking if value reaches zero.
6. sem_post() increments the semaphore, waking blocked threads.
7. Producer waits on empty_slots before adding items to the buffer.
8. Consumer waits on full_slots before removing items from the buffer.
9. Larger buffer sizes reduce contention and increase throughput.
10. With buffer size 5, throughput increased compared to buffer size 1.

Task 2: Deadlock Detection and Prevention

Question:

Implement a program that demonstrates a deadlock scenario where two threads attempt to acquire resources in a circular wait pattern. Analyze deadlock conditions and demonstrate prevention using resource ordering.

Source Code:

```
haasini@SSH: ~/OSSP_uplo
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

pthread_mutex_t resourceA = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resourceB = PTHREAD_MUTEX_INITIALIZER;

void alarm_handler(int sig)
{
    (void)sig;
    char msg[] =
        "\nDeadlock detected: both threads are waiting forever.\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
    _exit(0);
}

void *deadlock_thread1(void *arg)
{
    (void)arg;
    pthread_mutex_lock(&resourceA);
    printf("Thread 1 locked Resource A\n");
    sleep(1);
    printf("Thread 1 waiting for Resource B...\n");
    pthread_mutex_lock(&resourceB);
    return NULL;
}

void *deadlock_thread2(void *arg)
{
    (void)arg;
    pthread_mutex_lock(&resourceB);
    printf("Thread 2 locked Resource B\n");
    sleep(1);
    printf("Thread 2 waiting for Resource A...\n");
    pthread_mutex_lock(&resourceA);
    return NULL;
}

void *safe_thread(void *arg)
{
    long id = (long)arg;
    printf("Thread %ld waiting for Resource A\n", id);
    pthread_mutex_lock(&resourceA);
    printf("Thread %ld locked Resource A\n", id);
    pthread_mutex_lock(&resourceB);
    printf("Thread %ld locked Resource B\n", id);
    printf("Thread %ld completed work\n", id);
    pthread_mutex_unlock(&resourceB);
    pthread_mutex_unlock(&resourceA);
    return NULL;
}

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: %s deadlock|prevent\n", argv[0]);
        return 1;
    }

    pthread_t t1, t2;
    if (strcmp(argv[1], "deadlock") == 0)
    {
        printf("=== DEADLOCK DEMONSTRATION ===\n");
        signal(SIGALRM, alarm_handler);
        alarm(0);
        pthread_create(&t1, NULL, deadlock_thread1, NULL);
        pthread_create(&t2, NULL, deadlock_thread2, NULL);
        pthread_join(t1, NULL);
        pthread_join(t2, NULL);
    }
    else if (strcmp(argv[1], "prevent") == 0)
    {
        printf("=== DEADLOCK PREVENTION ===\n");
        printf("Both threads acquire resources in A -> B order.\n");
        pthread_create(&t1, NULL, safe_thread, (void *)1);
        pthread_create(&t2, NULL, safe_thread, (void *)2);
        pthread_join(t1, NULL);
        pthread_join(t2, NULL);
        printf("Both threads completed successfully.\n");
        printf("Deadlock prevented using resource ordering.\n");
    }
    else
    {
        printf("Invalid mode.\n");
    }
    return 0;
}
```

Output:

Deadlock Demonstration

```
haasini@SSH: ~/OSSP_upload/Practical_Week12$ gcc -Wall -Wextra P12_T2.c
haasini@SSH:~/OSSP_upload/Practical_Week12$ ./P12_T2 deadlock
=== DEADLOCK DEMONSTRATION ===
Thread 1 locked Resource A
Thread 2 locked Resource B
Thread 2 waiting for Resource A...
Thread 1 waiting for Resource B...

Deadlock detected: both threads are waiting forever.
haasini@SSH:~/OSSP_upload/Practical_Week12$ |
```

Deadlock Prevention – Resource Ordering

```
Deadlock detected. Both threads are waiting forever.  
haasini@SSH:~/OSSP_upload/Practical_Week12$ ./P12_T2 prevent  
=== DEADLOCK PREVENTION ===  
Both threads acquire resources in A -> B order.  
  
Thread 1 waiting for Resource A  
Thread 1 locked Resource A  
Thread 1 locked Resource B  
Thread 1 completed work  
Thread 2 waiting for Resource A  
Thread 2 locked Resource A  
Thread 2 locked Resource B  
Thread 2 completed work  
  
Both threads completed successfully.  
Deadlock prevented using resource ordering.  
haasini@SSH:~/OSSP_upload/Practical_Week12$ |
```

Observations:

1. Deadlock requires four necessary conditions: (1) Mutual Exclusion, (2) Hold and Wait, (3) No Preemption, (4) Circular Wait.
2. In the demonstration, Thread 1 acquires Resource A and waits for Resource B while holding A.
3. Thread 2 acquires Resource B and waits for Resource A while holding B.
4. This creates a circular wait where neither thread can proceed indefinitely.
5. Breaking the circular wait prevents deadlock by enforcing consistent resource ordering.
6. In prevention mode, both threads acquire resources in the same A-then-B order.
7. With consistent ordering, Thread 1 acquires A, then B, then completes work.
8. Thread 2 is forced to wait for Thread 1 to release A before acquiring it.
9. No circular wait exists when all threads request resources in the same order.
10. Deadlock prevention through resource ordering demonstrates eliminating the circular-wait condition.